Data Structures (ETCCDS202)

Unit–4 Assignment: Data Management Mini Toolkit (DMMT)

Course Details

Name of the School: School of Engineering & Technology

Program: B.Tech CSE (AI ML)

Course Title: Data Structures

Course Code: ETCCDS202

Unit Title: Non-linear and advanced data structures

Student Name: Ankit Kumar Mishra

Roll No.: 2501730079

Section: A

Introduction

In this assignment, I have implemented three important data structures: Binary Search Tree (BST), Graph, and Hash Table using Python. These data structures are very useful for organizing, storing, and searching data efficiently in computer programs.

BST helps in maintaining data in sorted order and allows fast searching operations. Graph is used to represent connections between different elements and helps in traversal using BFS and DFS algorithms. Hash Table provides fast access to data using key-value pairs and handles collisions using separate chaining.

This assignment helped me understand the basic working and real-life importance of these data structures.

1. Binary Search Tree (BST)

Why inorder traversal prints sorted data

In a Binary Search Tree, all the values in the left subtree are smaller than the root node and all the values in the right subtree are greater than the root node.

In inorder traversal, we follow the order:
Left → Root → Right

So first all smaller elements are printed, then root, then larger elements.
That is why the output comes in sorted order.

Time Complexity (Simple Explanation)

Average Case:
Time complexity is O(log n)
Because the tree is balanced and we move half of the tree each time.

Worst Case:
Time complexity is O(n)
This happens when the tree becomes like a straight line (skewed tree), for example inserting sorted data.

2. Graph (BFS and DFS)

Difference between BFS and DFS

BFS (Breadth First Search):
It visits nodes level by level.
It uses a queue.

DFS (Depth First Search):
It goes deep into one path first and then comes back.
It uses recursion or stack.

Real-world Use Cases

BFS:
Used for finding shortest path in an unweighted graph
Example: finding shortest route in a map.

DFS:
Used for cycle detection
Example: checking loops in networks or circuits.

3. Hash Table (Separate Chaining)

What is a Collision?

A collision happens when two different keys get the same index after applying the hash function.

Example:
If table size = 5
10 % 5 = 0
15 % 5 = 0

So both go to same index → collision occurs.

Why Separate Chaining Helps

In separate chaining, we store multiple elements at the same index using a list.

So even if collision happens, we can store all values properly without losing data.

Time Complexity

Average Case:
O(1) → very fast (direct access)

Worst Case:
O(n) → if many elements go to same index (too many collisions)

Conclusion

In this assignment, we implemented three important data structures:

- BST for ordered data and searching
- Graph for traversal using BFS and DFS
- Hash Table for fast key-value storage

These structures are widely used in real-life applications like databases, searching systems, and networking.